2043. Simple Bank System

Solved

Medium | Topics | Companies | Hint

You have been tasked with writing a program for a popular bank that will automate all its incoming transactions (transfer, deposit, and withdraw). The bank has `n` accounts numbered from `1` to `n`. The initial balance of each account is stored in a **0-indexed** integer array `balance`, with the `(i + 1)`th account having an initial balance of `balance[i]`.

Execute all the **valid** transactions. A transaction is **valid** if:

- The given account number(s) are between `1` and `n`, and
- The amount of money withdrawn or transferred from is **less than or equal** to the balance of the account.

Implement the `Bank` class:

- `Bank(long[] balance)` Initializes the object with the **0-indexed** integer array `balance`.
- `boolean transfer(int account1, int account2, long money)` Transfers `money` dollars from the account numbered `account1` to the account numbered `account2`. Return `true` if the transaction was successful, `false` otherwise.
- `boolean deposit(int account, long money)` Deposit `money` dollars into the account numbered `account`. Return `true` if the transaction was successful, `false` otherwise.
- `boolean withdraw(int account, long money)` Withdraw `money` dollars from the account numbered `account`. Return `true` if the transaction was

Code

Java | Auto

```java
class Bank {

    long[] bal;
    int n;

    public Bank(long[] balance) {
        bal = balance;
        n = balance.length;
    }

    private boolean valid(int acc) {
        return acc >= 1 && acc <= n;
    }

    public boolean transfer(int account1, int account2, long money) {
        if (!valid(account1) || !valid(account2)) return false;
        if (bal[account1 - 1] < money) return false;

        bal[account1 - 1] -= money;
        bal[account2 - 1] += money;
        return true;
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 2 ms

Case 1

Input

```
["Bank","withdraw","transfer","deposit","transfer","withdraw"]
```

Description | Editorial | Solutions | Submissions

355. Design Twitter

Solved

Medium | Topics | Companies

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the `10` most recent tweets in the user's news feed.

Implement the `Twitter` class:

- `Twitter()` Initializes your twitter object.
- `void postTweet(int userId, int tweetId)` Composes a new tweet with ID `tweetId` by the user `userId`. Each call to this function will be made with a unique `tweetId`.
- `List<Integer> getNewsFeed(int userId)` Retrieves the `10` most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user themself. Tweets must be **ordered from most recent to least recent**.
- `void follow(int followerId, int followeeId)` The user with ID `followerId` started following the user with ID `followeeId`.
- `void unfollow(int followerId, int followeeId)` The user with ID `followerId` started unfollowing the user with ID `followeeId`.

Example 1:

Input
```
["Twitter", "postTweet", "getNewsFeed", "follow",
```

4.7K 201 80 Online

Code

Java | Auto

```java
import java.util.*;

class Twitter {

    private static int time = 0;

    // user -> set of followees
    private Map<Integer, Set<Integer>> followMap;

    // user -> list of tweets [tweetId, time]
    private Map<Integer, List<int[]>> tweetMap;

    public Twitter() {
        followMap = new HashMap<>();
        tweetMap = new HashMap<>();
    }

    public void postTweet(int userId, int tweetId) {
        followMap.putIfAbsent(userId, new HashSet<>());
        tweetMap.putIfAbsent(userId, new ArrayList<>());
        tweetMap.get(userId).add(new int[]{tweetId, time++});
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 6 ms

Case 1

Input

```
["Twitter","postTweet","getNewsFeed","follow","postTweet","getNewsFeed","unfollow","getNewsFeed"]
```

535. Encode and Decode TinyURL

Solved

Medium | Topics | Companies

> Note: This is a companion problem to the System Design problem: Design TinyURL.

TinyURL is a URL shortening service where you enter a URL such as `https://leetcode.com/problems/design-tinyurl` and it returns a short URL such as `http://tinyurl.com/4e9iAk`. Design a class to encode a URL and decode a tiny URL.

There is no restriction on how your encode/decode algorithm should work. You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can be decoded to the original URL.

Implement the `Solution` class:

- `Solution()` Initializes the object of the system.
- `String encode(String longUrl)` Returns a tiny URL for the given `longUrl`.
- `String decode(String shortUrl)` Returns the original long URL for the given `shortUrl`. It is guaranteed that the given `shortUrl` was encoded by the same object.

Example 1:

> **Input:** `url = "https://leetcode.com/problems/design-tinyurl"`
> **Output:** `"https://leetcode.com/problems/design-tinyurl"`


```java
public class Codec {

    // Encodes a URL to a shortened URL.
    public String encode(String longUrl) {
        return longUrl;

    }

    // Decodes a shortened URL to its original URL.
    public String decode(String shortUrl) {
        return shortUrl;

    }
}

// Your Codec object will be instantiated and called as such:
// Codec codec = new Codec();
// codec.decode(codec.encode(url));
```

View less

Testcase | Test Result

You must run your code first